



Team : Water Rangers

Water Quality Prediction

Python Project Report

CPE393 : Introduction to Data Science with Python

Contributors :

Nicolas CORNU LAPORTE

Athenais MENDY

Alix GALANT

July 6, 2026

Table of contents

1	Introduction	2
2	Problem Definition and Goals	2
2.1	Context	2
2.2	Problem Statement	2
2.3	Proof of Concept "River Ganga Case Study"	3
2.4	Mission and Technical Approach	3
2.4.1	Analysis and Training Phase (Notebook)	4
2.4.2	Deployment Phase (Web Application)	4
2.5	Project architecture	4
3	Dataset Description	5
4	Use Machine learning to solve the problem	6
4.1	Data Visualization and EDA	8
4.2	Preprocessing	12
4.2.1	Line-by-Line Explanation of the Code	13
4.3	Machine learning Models	14
4.3.1	Why We Chose These Models and Their Parameters	14
4.4	Models Training	15
4.4.1	Step-by-Step Explanation of the Training Loop	16
5	Evaluation Metrics and Results	17
6	Responsible AI Practices	18
6.1	Analyzing the Modest Performance: Why are the Scores around 66% to 68%?	19
6.2	Privacy Issues, Ethical Risks, and Avoiding Misleading Claims	20
7	Feature Engineering (Web GUI)	21
7.1	Model Serialization and Export	21
7.2	Deployment Phase: Streamlit	21
7.3	Web Application Architecture	21
7.4	Application Logic	22
7.5	Future Perspectives	25
8	Conclusion and Future Work	26
9	Team Contribution	26
	Bibliographie	27

1 Introduction

In our modern societies, access to clean drinking water is often taken for granted as a basic health and technological standard. However, ensuring water safety remains one of the most complex engineering and public health challenges of our century. As computer science students, we often tend to focus on purely digital problems. Yet, the current rise of Data Science and Machine Learning gives us the tools to interact directly with the physical world and solve real-life crises.

This project sits exactly at the intersection of applied computer science and environmental engineering. Our goal is to develop a Machine Learning model in Python that can classify and predict a sample's Potability based on its physicochemical properties (pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic carbon, Trihalomethanes, and Turbidity). More than just a simple coding exercise, this work aims to build a decision-support tool designed to overcome the limits of traditional water testing methods.

2 Problem Definition and Goals

2.1 Context

To understand the relevance of an Artificial Intelligence approach, it is necessary to place this project in its global, national, and institutional context.

On a global scale, the situation remains critical. Joint data from the WHO and UNICEF highlight that over 2.1 billion people still lack access to safely managed water, and 115 million still consume untreated surface water [1]. Faced with this reality, the United Nations (UN) has made access to water and sanitation its 6th Sustainable Development Goal (SDG 6).

In Thailand, our host country, the issue is somewhat paradoxical. The main challenge is no longer the water supply infrastructure, but rather the final quality of the resource. The Thai government reports a water access rate of 99.5%. However, reports from the UN and the Ministry of Health reveal a major health gap: only 40.8% of the water available in rural households is actually safe for direct consumption [2]. Nearly 15.5% of this water is heavily contaminated by industrial or agricultural waste (heavy metals, pesticides), requiring extensive treatment before drinking.

This complex local context perfectly aligns with the vision of King Mongkut's University of Technology Thonburi (KMUTT). Known for its strong commitment to sustainable development (notably through its excellent results in the UI GreenMetric ranking), KMUTT does not just apply SDG 6 on its own campuses. It also encourages its students to develop affordable and accessible technologies for local communities [3]. This project is a direct reflection of this academic philosophy: using computer science to turn raw data into a positive social impact.

2.2 Problem Statement

Despite the urgency of the situation, field actors (NGOs, humanitarian organizations, local governments) face a major technical and logistical obstacle: how can we verify if water is safe to drink when located in an isolated or low-resource area?

Traditional laboratory testing methods are highly accurate, but they suffer from three structural flaws that make them unsuited for emergency situations or remote areas [4].

First, the financial cost is a major issue. A complete certified analysis can cost between \$100 and \$400 (around 3,500 to 14,000 Baht) per sample. For an NGO monitoring dozens of wells, this budget quickly becomes a roadblock.

Second, there is a significant time bottleneck. Testing for pathogens (like *E. coli*) requires a mandatory incubation period in the lab, usually taking 24 to 72 hours. On top of that, labs often process samples in batches, which can push the wait time for results to 5 or even 14 days. During this turnaround time, local populations continue to consume a potentially deadly resource.

Finally, logistical constraints make the process even harder. Transporting a sample requires maintaining a strict cold chain. For example in Thai border regions or isolated villages, this condition is almost impossible to meet.

Therefore, our main problem can be formulated as follows: In a context where lab analyses are too slow, too expensive, and logistically unsuited for isolated areas, how can Artificial Intelligence use instant and low-cost physicochemical readings to predict water safety with enough reliability to support decision-making?

2.3 Proof of Concept "River Ganga Case Study"

To illustrate the viability of this approach, we can look at a recent applied study conducted on the River Ganga in Haridwar, India [5]. Researchers successfully utilized Machine Learning techniques to accurately predict the Water Quality Index (WQI) of the river based on historical data. By analyzing a comprehensive dataset of accessible physicochemical parameters, such as pH, Dissolved Oxygen (DO), Biochemical Oxygen Demand (BOD), and total coliform, the study demonstrated that AI algorithms can efficiently map complex, non-linear environmental data. The research evaluated multiple predictive models, with top-performing algorithms achieving a highly reliable prediction accuracy (an R^2 score of 0.9655) with minimal error margins. Ultimately, this research highlights how machine learning can serve as a precise and proactive tool for water quality monitoring.

This success story serves as a foundational proof of concept for our project, but our ambition is to globalize this approach. While the Indian project was a localized trial, the logic of this "triage system" is universally applicable. Our goal with the WaterRangers project is to scale this methodology beyond specific regional pilot programs by creating a standardized, open-source tool. By making this predictive framework accessible to any community, regardless of their location, we aim to bridge the gap between AI research and immediate local needs, transforming a specialized solution into a global resource.

2.4 Mission and Technical Approach

Our mission, led by our WaterRangers team, is to build a bridge between Data Science and immediate humanitarian needs. Rather than trying to replace bacteriological testing labs, we aim to create a predictive, fast, accessible, and open-source "triage system".

Today, it is technologically simple and inexpensive to collect field metrics (pH, hardness, solids, chloramines, sulfates, conductivity, organic carbon, trihalomethanes, turbidity). Taken together, these variables form a complex signature that a machine learning tool can decipher. To leverage this data, the work of the WaterRangers team is technically structured around two main axes:

2.4.1 Analysis and Training Phase (Notebook)

The analytical core of the project will rely on a Notebook environment. We will start with the most comprehensive and community-recognized dataset on the Kaggle platform. Our technical pipeline will consist of:

1. Cleaning and exploring (Exploratory Data Analysis) this dataset to understand the distribution and correlation of the physicochemical variables.
2. Training and comparing 3 distinct Machine Learning models ("random-forest, Gradient Boosting, XG Boost").
3. Exporting these 3 models once optimized and trained, using the joblib library to ensure they remain persistent and lightweight.

2.4.2 Deployment Phase (Web Application)

An AI model only has an impact if it is usable by field workers. The second part of our project therefore consists of encapsulating our 3 pre-trained models into a simple, user-friendly web application developed using the Streamlit Python framework.

True to the values of sharing promoted by KMUTT, our WaterRangers solution will be fully open source. The software architecture is designed to be hybrid: the application could eventually be hosted on the cloud for global accessibility, but most importantly, it offers the ability to run 100% autonomously locally with the embedded models. In fact, it is in this local configuration, which is ideal for offline areas without an internet connection, that the project will be deployed for our university demonstration.

2.5 Project architecture

The entire project is available as open source on a public GitHub repository. (link : <https://github.com/nikor3/WaterRangers>). This approach ensures free access to our solution for field workers while allowing for contributions and peer review. The repository is structured pragmatically, separating the research phase from the software deployment phase.

At the root of the repository, you will find:

1. The README.md file: This serves as the central technical documentation. It details how to set up Python virtual environments and explains step-by-step how to run the project. It covers both how to launch the Notebook (using local solutions like Jupyter or cloud platforms like Google Colab) and how to start the web application.
2. The Jupyter Notebook: The file containing the Exploratory Data Analysis (EDA), the dataset preparation, and the training of the three Machine Learning models.
3. This report: The document you are currently reading, which summarizes the context, technical choices, and analysis of the results.

The /app folder is dedicated exclusively to the deployment of the Streamlit web application. It is designed to run 100% autonomously in a local environment, which is an absolute necessity for regions without internet access. It includes:

1. The source code for the user interface.

2. The pre-trained Machine Learning models (exported using the joblib library).
3. The assets (visual resources and any additional files needed for the interface).

3 Dataset Description

The dataset chosen is issued from Kaggle’s dataset bank under the name “Water Quality”, and uploaded by the user Aditya Kadiwal. It contains a `water_potability.csv` file that features all the data necessary to exploit the information about multiple bodies of water. It contains water quality metrics for 3,276 different water bodies.

URL: <https://www.kaggle.com/datasets/adityakadiwal/water-potability/>

Our dataset is under the CC0 1.0 Universal Public Domain Dedication. Re-users are granted full, unrestricted permission to copy, modify, distribute, and perform the work, even for commercial purposes, without asking for permission. It is split into 10 different columns and 3,276 rows. Each column represents the following parameters/characteristics describing the 3,276 bodies of water recorded in the dataset: pH, hardness, solids, chloramines, sulphate, conductivity, organic carbon, trihalomethanes, and turbidity.

Dataset Features

1. **pH value:** pH is an important parameter in evaluating the acid - base balance of water. It is also the indicator of acidic or alkaline condition of water status. WHO has recommended a maximum permissible limit of pH from 6.5 to 8.5. The current investigation ranges were 6.52 to 6.83, which are within the range of WHO standards.
2. **Hardness:** Hardness is mainly caused by calcium and magnesium salts. These salts are dissolved from geologic deposits through which water travels. The length of time water is in contact with hardness-producing material helps determine how much hardness there is in raw water. Hardness was originally defined as the capacity of water to precipitate soap caused by Calcium and Magnesium.
3. **Solids (Total Dissolved Solids - TDS):** Water has the ability to dissolve a wide range of inorganic and some organic minerals or salts such as potassium, calcium, sodium, bicarbonates, chlorides, magnesium, sulfates, etc. These minerals produce an unwanted taste and diluted color in the appearance of water. This is an important parameter for the use of water. Water with a high TDS value indicates that the water is highly mineralized. The desirable limit for TDS is 500 mg/L and the maximum limit is 1,000 mg/L, which is prescribed for drinking purposes.
4. **Chloramines:** Chlorine and chloramine are the major disinfectants used in public water systems. Chloramines are most commonly formed when ammonia is added to chlorine to treat drinking water. Chlorine levels up to 4 milligrams per liter (mg/L or 4 parts per million (ppm)) are considered safe in drinking water.
5. **Sulfate:** Sulfates are naturally occurring substances that are found in minerals, soil, and rocks. They are present in ambient air, groundwater, plants, and food. The

principal commercial use of sulfate is in the chemical industry. Sulfate concentration in seawater is about 2,700 mg/L. It ranges from 3 to 30 mg/L in most freshwater supplies, although much higher concentrations (1,000 mg/L) are found in some geographic locations.

6. **Conductivity:** Pure water is not a good conductor of electric current; rather, it is a good insulator. An increase in ion concentration enhances the electrical conductivity of water. Generally, the amount of dissolved solids in water determines the electrical conductivity. Electrical conductivity (EC) actually measures the ionic process of a solution that enables it to transmit current. According to WHO standards, the EC value should not exceed 400 $\mu\text{S}/\text{cm}$.
7. **Organic Carbon:** Total Organic Carbon (TOC) in source waters comes from decaying natural organic matter (NOM) as well as synthetic sources. TOC is a measure of the total amount of carbon in organic compounds in pure water. According to the US EPA, it should be < 2 mg/L as TOC in treated/drinking water, and < 4 mg/L in source water used for treatment.
8. **Trihalomethanes:** THMs are chemicals that may be found in water treated with chlorine. The concentration of THMs in drinking water varies according to the level of organic material in the water, the amount of chlorine required to treat the water, and the temperature of the water being treated. THM levels up to 80 ppm are considered safe in drinking water.
9. **Turbidity:** The turbidity of water depends on the quantity of solid matter present in the suspended state. It is a measure of the light-emitting properties of water, and the test is used to indicate the quality of waste discharge with respect to colloidal matter. The mean turbidity value obtained for the Wondo Genet Campus (0.98 NTU) is lower than the WHO recommended value of 5.00 NTU.
10. **Potability:** Indicates if water is safe for human consumption, where 1 means Potable and 0 means Not potable.

Target Variable Selection

For the predictive modeling phase of this project, **Potability** was selected as the target variable. As a major metric of water safety, analyzing potability allows us to understand how different physicochemical water properties interact to determine safety for human consumption. Predicting this feature provides a meaningful benchmark for evaluating the predictive power of the remaining features.

4 Use Machine learning to solve the problem

Access to clean and safe drinking water is vital for human health. However, checking if water is safe (potable) can be difficult because many different factors are involved, such as chemical levels, acidity, and clarity. Traditionally, this requires long and expensive laboratory tests.

To solve this problem, we can use Machine Learning. Machine learning allows us to build a smart computer model that looks at data from past water samples and automatically learns the rules to separate safe water from unsafe water.

In terms of computer science, this is a Binary Classification problem. The model receives 9 different measurements from a water sample (like pH, Sulfate, or Turbidity) and must choose between two possible outcomes:

- **Target = 1 (Potable):** The water is safe for humans to drink.
- **Target = 0 (Not Potable):** The water is contaminated and unsafe.

Using an AI model for this task is incredibly useful. Once the model is properly trained, it can give a safety result in milliseconds. This allows us to create a fast web application where users can type in their water parameters and instantly find out if the water is dangerous or safe.

To successfully solve this problem from start to finish, our code follows a clear 4-step workflow, which we will detail in the following subsections:

1. **Import and clean the data:** First, we fetch our water dataset and fix any errors or missing information so that our models have perfect data to learn from.
2. **Machine learning Algorithms:** Next, we select and set up three powerful modern algorithms based on decision trees (Random Forest, Gradient Boosting, and XGBoost) to compare their intelligence.
3. **How to train models ?** Then, we let the algorithms study our clean data so they can find the hidden patterns and optimize their formulas.
4. **How to predict results ?** Finally, we save the brains of our trained models into static files and connect them to our web application to test real-world scenarios.

First, let's have a look at the dataset's visualization:

4.1 Data Visualization and EDA

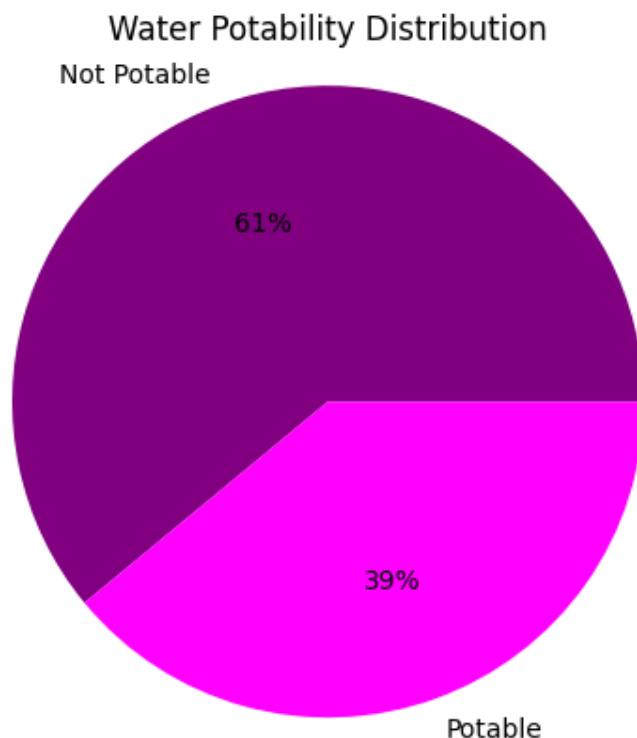


Figure 1: Distribution of Potable (1) vs. Non-Potable (0) water bodies in the dataset.

Figure 1 illustrates the distribution of the binary target variable, **Potability**. Non-potable water bodies constitute the majority of the dataset at 61%, while potable water bodies account for the remaining 39%. This reveals a moderate class imbalance. While it is not severe enough to completely stall model training, it indicates that standard classification accuracy could be a misleading metric. Consequently, performance evaluation should rely heavily on metrics like Precision, Recall, and the F1-Score to ensure the models do not simply favor the majority class.

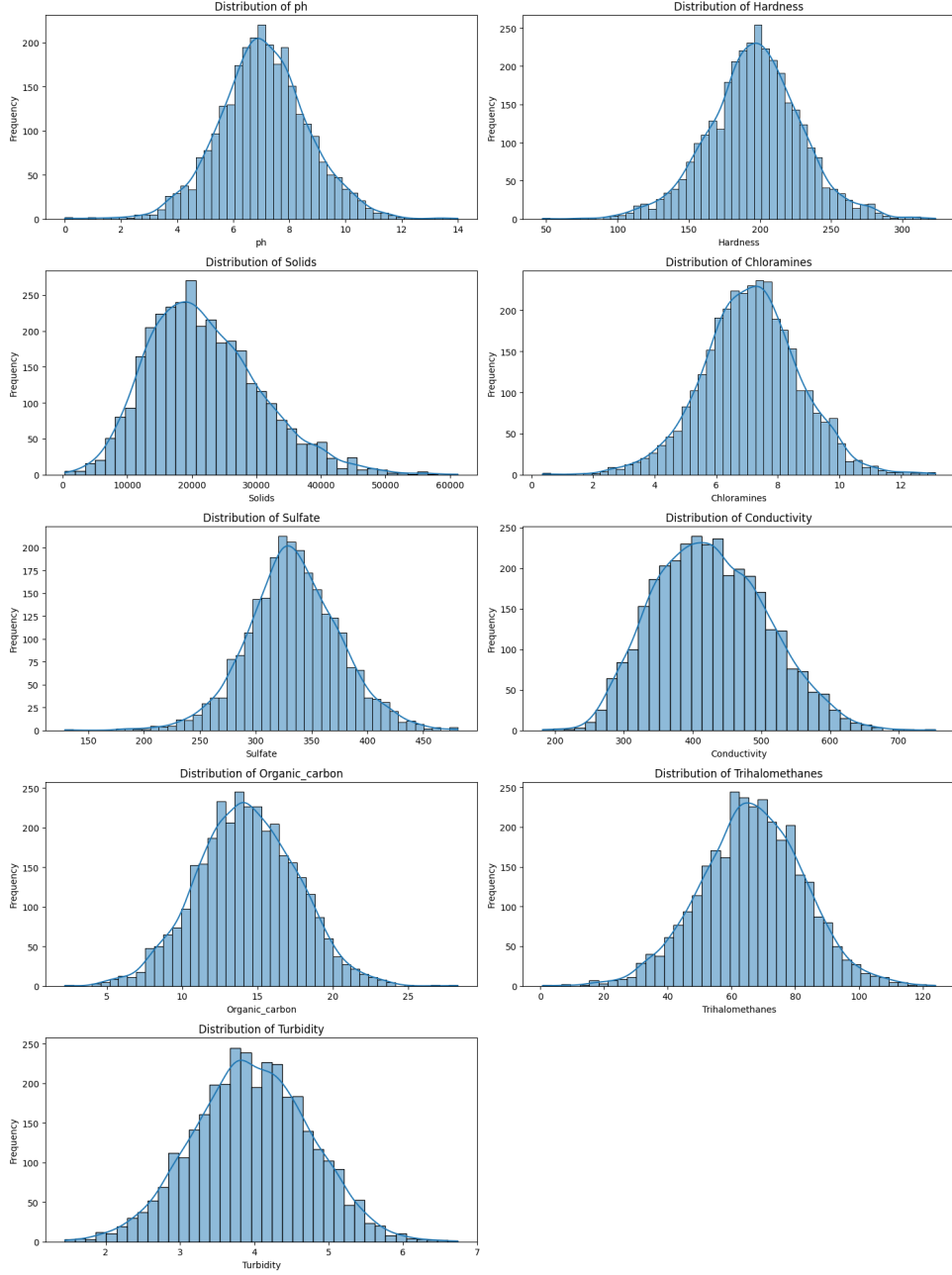


Figure 2: Histograms showing the frequency distribution of each water quality parameter.

Figure 2 shows the individual histograms overlaid with Kernel Density Estimate (KDE) curves for all 9 continuous predictive features. Most features—specifically `pH`, `Hardness`, `Chloramines`, `Sulfate`, `Conductivity`, `Trihalomethanes`, and `Turbidity`—exhibit near-perfect Gaussian (normal) distributions. Conversely, `Solids` displays a notable right-skewed distribution. The clean, symmetric profile of the majority of these independent variables suggests that advanced data transformations (like log scaling) may only be minimally necessary, though standardization will still be required for distance-based models.

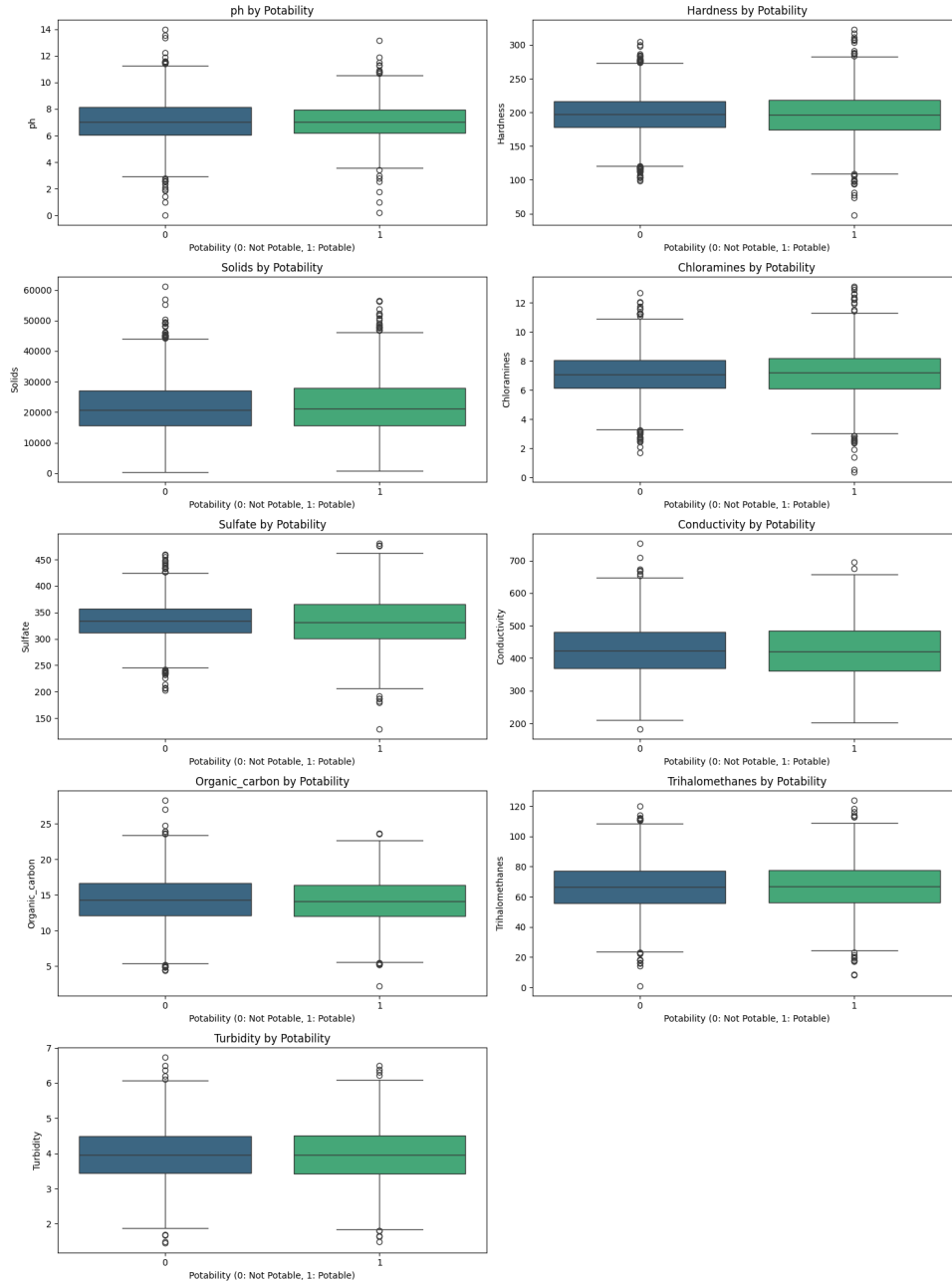


Figure 3: Distribution of each parameter by potability (0: non-potable, 1: potable)

Figure 3 contrasts the distribution of each feature against potable (1) and non-potable (0) states. Crucially, the median, interquartile ranges (IQR), and overall distribution spreads are nearly identical across both classes for every single feature. There is a prominent presence of outliers across all variables, yet none show a clear visual shift in baseline values based on whether the water is safe to drink. This implies that no single feature acts as a "silver bullet" predictor, confirming that a multivariate machine learning approach is required to capture subtle non-linear boundaries.

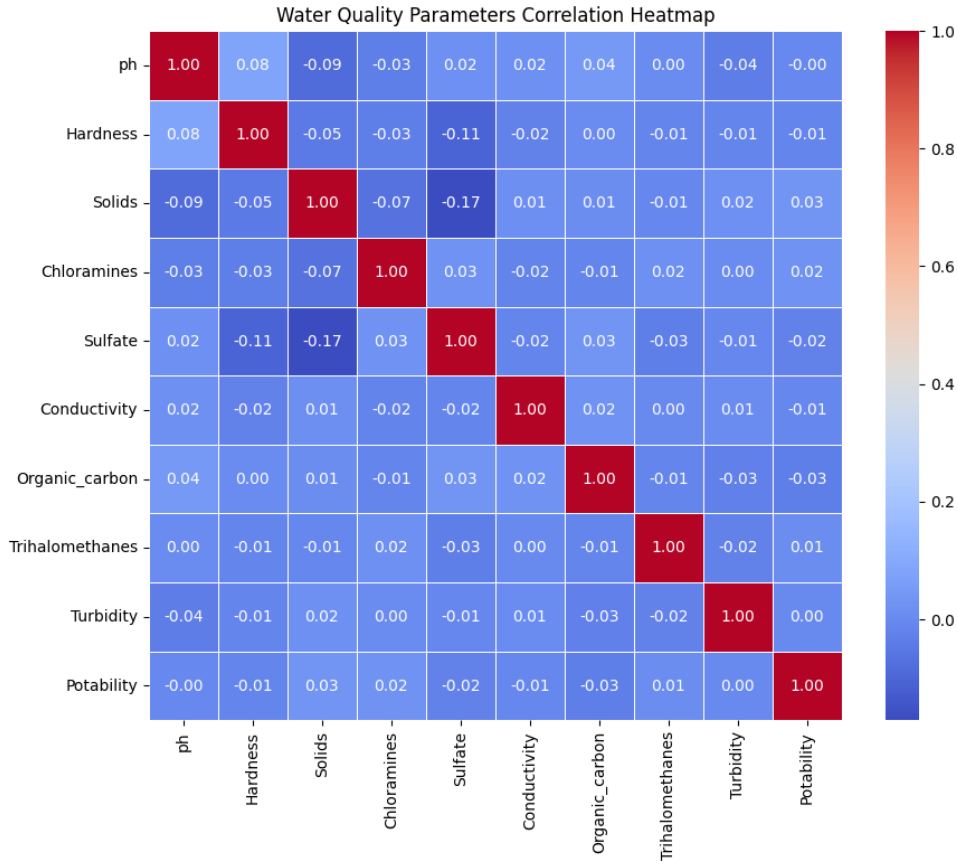


Figure 4: Pearson correlation coefficients between water quality parameters

Figure 4 contains the Pearson correlation matrix calculated across all attributes. The highest linear associations observed are incredibly weak, such as the negative correlation between **Sulfate** and **Solids** (-0.17) and a slight positive correlation between **Hardness** and **ph** (0.08). All correlations with the target variable **Potability** sit hovering around ± 0.01 . This definitive lack of linear correlation underscores two structural facts: first, multicollinearity is completely absent, meaning feature reduction methods like PCA are unnecessary; second, linear models (like standard Logistic Regression) will likely struggle, motivating the use of non-linear tree-based ensembles.

Missing Values in the dataset

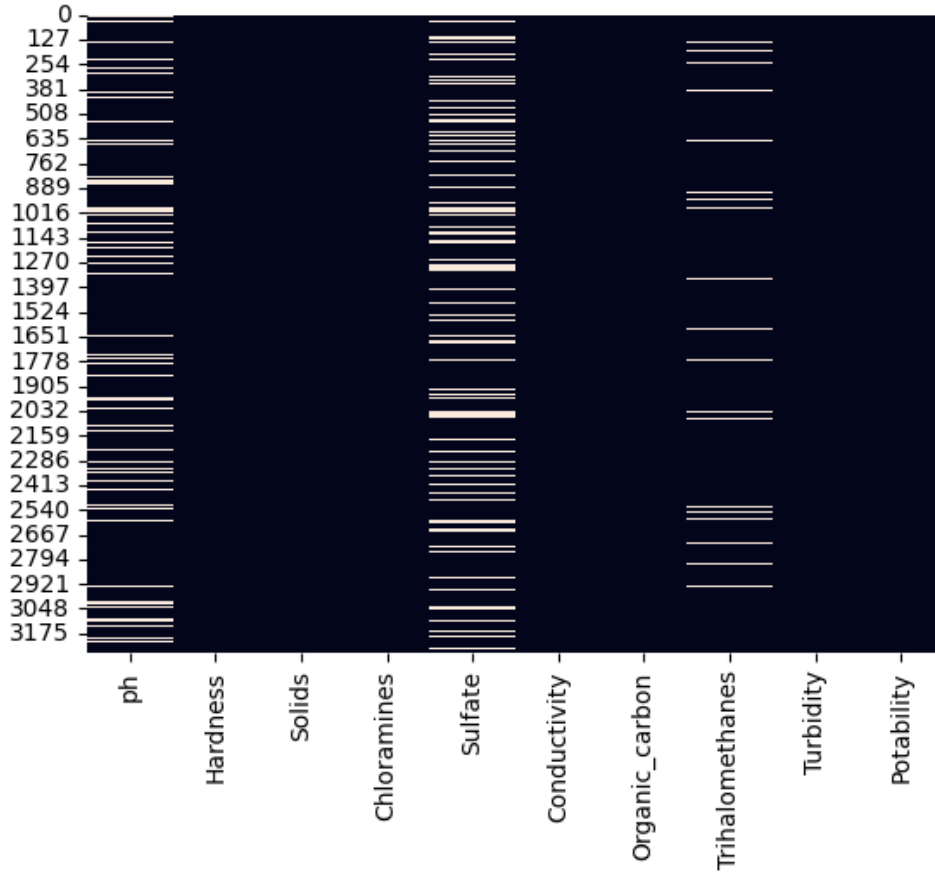


Figure 5: Missing values distribution in the dataset

Figure 5 illustrates the distribution and density of missing values across the dataset prior to cleaning. We can observe that the missing data is not uniform: it is strictly localized within three specific parameters: `ph`, `Sulfate`, and `Trihalomethanes`. The remaining six features, as well as the target variable `Potability`, are completely intact with zero missing entries. This visual diagnosis confirms that the data is Missing At Random (MAR), which mathematically justifies our choice of using a column-wise median imputation strategy (`fillna(X.median())`) rather than deleting rows, thereby preserving the structural integrity of our 3,276 samples.

Now, let us explore each step in detail to see how the code works.

4.2 Preprocessing

Before we can train any machine learning algorithm, we need to prepare our data. A machine learning model is only as good as the data we give it. If the data is messy or full of mistakes, the predictions will be wrong. This is why data preprocessing is one of the most important steps in data science.

For this project, our data comes from a popular public dataset containing 3,276 water samples. Each sample has 9 metrics (like pH, Hardness, and Turbidity) and a label showing if it is potable or not.

Here is the exact Python code used in our notebook to download, import, and clean the water data:

```
1 # Download latest version
2 path = kagglehub.dataset_download("adityakadiwal/water-potability")
3
4 # Import datasets / DataFrame
5 df_train = pd.read_csv(path + "/water_potability.csv")
6
7 # Pre-processing
8 X = df_train.drop(columns=['Potability'])
9 y = df_train['Potability']
10
11 # Filling NaN values
12 X = X.fillna(X.median())
13
14 # Use 80% train, and 20% secret validation (with stratification to
   preserve class balance)
15 X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
   random_state=42, stratify=y)
```

4.2.1 Line-by-Line Explanation of the Code

- `path = kagglehub.dataset_download("adityakadiwal/water-potability")`
This line connects to the internet and downloads the raw water dataset directly from Kaggle using the `kagglehub` tool. It saves the files on your computer and stores the folder path inside a variable called `path`.
- `df_train = pd.read_csv(path + "/water_potability.csv")`
This line takes the raw CSV file from our computer and opens it using Pandas (`pd.read_csv`). It transforms the file into a clean, virtual table with rows and columns, called a DataFrame (`df_train`), so we can manipulate it with Python.
- `X = df_train.drop(columns=['Potability'])`
Here, we create our feature matrix `X`. We take the full table and drop (remove) the `Potability` column. Now, `X` contains only the 9 chemical and physical measurements (like pH and Sulfate) that the model will use to analyze the water.
- `y = df_train['Potability']`
Here, we create our target vector `y`. We select *only* the `Potability` column. This column contains the correct answers (0 for unsafe, 1 for safe). This is what our model will try to guess during the test.
- `X = X.fillna(X.median())`
Since algorithms cannot process empty boxes, this line automatically find every empty spot and fill them with the median (the middle value) of that specific column. This is a very safe way to clean the data without deleting important rows.
- `X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`
This is the final split, and it does multiple things at once:
 - It divides the data into two parts: 80% goes to the training sets (`X_train`, `y_train`) for learning, and 20% goes to the validation sets (`X_val`, `y_val`) to act as a secret exam.

- `test_size=0.2` specifies that exactly 20% is kept for the validation exam.
- `random_state=42` is just a seed number. It ensures that every time we run the code, the data is mixed in the exact same way, so our results don't change randomly.
- `stratify=y` is highly important. It forces the 80/20 split to keep the exact same balance of safe and unsafe water in both groups. This prevents the model from becoming biased toward one answer.

4.3 Machine learning Models

Once our water dataset is clean and divided, we need to choose the mathematical brains that will analyze it. Because our target is binary (0 or 1), we must specifically use Classification Algorithms. The job of a classifier is to draw a boundary line between different groups of data to separate them.

Instead of relying on just one algorithm, we decided to use three different models. This is a standard practice in data science because no single algorithm is perfect for every dataset. By using three distinct models, we can compare their performance, find out which one understands the water parameters best, and avoid making a biased choice.

Here is the exact Python code used to initialize our models and create a storage space for their scores:

```

1 # Models Dictionary
2 models = {
3     "Random Forest": RandomForestClassifier(n_estimators=300,
4     random_state=42),
5     "Gradient Boosting": GradientBoostingClassifier(n_estimators=150,
6     random_state=42),
7     "XGBoost": XGBClassifier(n_estimators=150, learning_rate=0.05,
8     random_state=42, eval_metric='logloss')
9 }
10
11 # Dictionary to store scores
12 results = {}

```

4.3.1 Why We Chose These Models and Their Parameters

All three algorithms we selected belong to a highly powerful family called **Tree-Based Ensembles**. Instead of using one giant decision tree, they combine many smaller trees together to make a final, accurate decision.

- **Random Forest Classifier:**

- *Why:* It creates hundreds of decision trees in parallel. Each tree gets a random subset of data and votes on whether the water is safe or unsafe. The majority vote wins. It is excellent because it is very stable and rarely overfits the data.
- *Parameters:*
 - * `n_estimators=300`: This tells the algorithm to build exactly 300 individual decision trees. More trees generally bring better stability.
 - * `random_state=42`: This locks the randomness so that the trees are built the exact same way every time we run the code.

- **Gradient Boosting Classifier:**

- *Why:* Unlike "Random Forest", this model builds trees one after the other. The second tree focuses entirely on correcting the mistakes made by the first tree, the third corrects the second, and so on. This makes it highly accurate at catching difficult water samples.
- *Parameters:*
 - * `n_estimators=150`: The model will build a sequence of 150 trees.
 - * `random_state=42`: Ensures reproducible and stable results.

- **XGBoost Classifier (Extreme Gradient Boosting):**

- *Why:* This is an advanced, ultra-fast version of Gradient Boosting. It uses smart mathematical equations to prevent the model from just memorizing the training data. It is currently one of the most successful algorithms in data science for tabular data.
- *Parameters:*
 - * `n_estimators=150`: It builds a chain of 150 sequential trees.
 - * `learning_rate=0.05`: This controls how fast the model learns. A smaller step size (0.05) means the model learns slowly and carefully, which prevents it from skipping past the best mathematical solution.
 - * `random_state=42`: Keeps the random splits consistent.
 - * `eval_metric='logloss'`: This is the internal scoring metric the algorithm uses during training to measure its own mistakes. "Logloss" heavily penalizes wrong predictions made with high confidence, forcing the model to become more accurate.

Finally, the line `results = {}` simply creates an empty Python dictionary. We will use this clean container in the next step to automatically store the final accuracy and performance scores of each model, making it easy to build a comparison chart later.

4.4 Models Training

Now that our models are configured and placed inside our dictionary, it is time for the most important phase: **Training**. Training means letting our algorithms study our training data (`X_train` and `y_train`) so they can find the hidden mathematical relationships between water characteristics and potability.

Instead of writing repetitive code for each of our three algorithms, we use a **Python for loop**. This loop automatically takes one model at a time, trains it, tests it on the hidden validation data, calculates its scores, and saves the results.

Here is the exact Python code responsible for executing this automated loop:

```
1 # Loop for Training and Evaluation
2 for name, model in models.items():
3     print(f"\n Training model : {name}")
4
5     # Training
6     model.fit(X_train, y_train)
7
8     # Predict
```



```

9     preds = model.predict(X_val)
10    probs = model.predict_proba(X_val)[: , 1]
11
12    # Compute Metrics
13    acc = accuracy_score(y_val, preds)
14    roc_auc = roc_auc_score(y_val, probs)
15
16    results[name] = {"Accuracy": acc, "ROC-AUC": roc_auc}

```

4.4.1 Step-by-Step Explanation of the Training Loop

Let us look at exactly what happens inside this loop for each model, line by line:

- **The Loop Initialization:**

The code for `name, model in models.items()`: sets up our loop. The `.items()` method goes through our dictionary and extracts two things at once: the nickname of the algorithm (like "XGBoost") stored in the variable `name`, and the actual algorithm object stored in the variable `model`.

- **The Fit Function:**

The code `model.fit(X_train, y_train)` is the exact moment where the machine learning happens. The `.fit()` function tells the algorithm to look at the water samples (`X_train`) along with the correct answers (`y_train`). The model adjusts its internal decision trees until it learns how to correctly guess if a sample is potable or not.

- **Hard Predictions:**

The code `preds = model.predict(X_val)` tests the model on our validation data (`X_val`) using `.predict()`. Crucially, the model does not get to see the correct answers (`y_val`) during this test. It gives its final hard guesses (0 or 1) for each sample, and we store them inside the variable `preds`.

- **Probability Predictions:**

The code `probs = model.predict_proba(X_val)[: , 1]` asks the model for its continuous probability percentage (for example: "I am 85% sure this water is safe"). The code `[: , 1]` extracts only the probability for the positive class (1, meaning Potable), which is a mandatory requirement to compute our advanced ROC-AUC metric.

- **Global Accuracy Calculation:**

The code `acc = accuracy_score(y_val, preds)` calculates our first metric: **Global Accuracy**. The function compares the model's guesses (`preds`) with the real hidden truths (`y_val`). It outputs a simple score representing the overall percentage of correct answers (e.g., 0.68 means 68% correct predictions).

- **ROC-AUC Calculation:**

The code `roc_auc = roc_auc_score(y_val, probs)` calculates our second metric: the **ROC-AUC Score**. Accuracy can sometimes be misleading if a dataset is unbalanced. ROC-AUC evaluates how good the model is at separating the safe samples from the dangerous ones across all possible probability levels. A score close to 0.5 means random guessing, while a score closer to 1.0 means perfect separation.

- **Saving the Scores:**

The code `results[name] = {"Accuracy": acc, "ROC-AUC": roc_auc}` saves our calculated scores. It takes the empty `results` dictionary we created earlier and adds a row for the current model using its `name` as the key. This allows us to keep a clean record of all scores so we can compare them and pick the absolute best model for our website.

5 Evaluation Metrics and Results

Once the models are trained and tested, we need to look at their performance. We must generate detailed metrics, structural matrices, and visual charts to deeply understand where our classifiers are succeeding and where they are failing.

This section of the code handles two things: first, it prints textual evaluation statistics inside the training loop for each individual model, and second, it builds a final graphical chart to compare all three models side-by-side.

```

1  # Results Output
2  print(f"--- Report Assessment for model {name} ---")
3  print(f"Global Accuracy : {acc:.4f}")
4  print(f"ROC-AUC score : {roc_auc:.4f}")
5  print("\nClassification Report :")
6  print(classification_report(y_val, preds))
7
8  # Print Confusion Matrix
9  cm = confusion_matrix(y_val, preds)
10 plt.figure(figsize=(4, 3))
11 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
12 plt.title(f"Confusion Matrix : {name}")
13 plt.xlabel("Predictions")
14 plt.ylabel("True values")
15 plt.show()
16
17 # FINAL COMPARISON
18 df_results = pd.DataFrame(results).T
19 print("\n=== FINAL RESULTS COMPARISON ===")
20 print(df_results)
21
22 # Performance Comparison Chart
23 df_results.plot(kind='bar', figsize=(10, 5))
24 plt.title('Accuracy VS ROC AUC (Tree-based Models Comparison)')
25 plt.ylabel('Score')
26 plt.xticks(rotation=0)
27 plt.ylim(0.4, 0.8)
28 plt.legend(loc='lower right')
29 plt.show()

```

- **The Classification Report:**

The code `print(classification_report(y_val, preds))` outputs an advanced statistics table from Scikit-Learn. It gives us three crucial values for each class (0 and 1):

- *Precision*: Out of all samples the model guessed as safe, how many were actually safe?

- *Recall*: Out of all truly safe water samples in the data, how many did the model manage to find?
- *F1-Score*: The harmonic balance between Precision and Recall.
- **Building the Confusion Matrix:**
The code `cm = confusion_matrix(y_val, preds)` calculates a grid that maps our correct versus incorrect classifications. It shows us exactly how many times the model got it right, and how many times it made an error (such as predicting water is safe when it is actually toxic).
- **Drawing the Heatmap Plot:**
The lines using `sns.heatmap()` and `plt.show()` transform the raw numbers of our confusion matrix into a colorful, blue-tinted graphical square. Setting `annot=True` writes the exact sample count numbers inside the squares, while `plt.xlabel` and `plt.ylabel` add clear text labels to our chart axes.
- **The Final Comparison Table:**
Outside of the loop, the line `df_results = pd.DataFrame(results).T` collects all saved dictionary scores and transforms them into a single summary table. The operation `.T` means "Transpose"; it flips our rows and columns so that our three models are listed cleanly as rows, and our metrics (*Accuracy* and *ROC-AUC*) are listed as columns.
- **The Performance Comparison Chart:**
The final block uses Matplotlib (`df_results.plot(kind='bar')`) to create a vertical bar chart. This chart displays both Accuracy and ROC-AUC scores side-by-side for all three models simultaneously. Setting `plt.ylim(0.4, 0.8)` sets the window scale of our vertical axis to focus specifically between 40% and 80%, allowing us to clearly see even small differences in performance so we can pick our champion model.

6 Responsible AI Practices

When we look at the final summary chart, we notice an interesting fact: even though our three models studied the exact same clean data (`X_train`), they do not have the same Accuracy or ROC-AUC scores. For some specific water samples, one model might guess "Safe" while another model guesses "Unsafe".

This happens because each algorithm has a unique internal mathematical structure and uses a different strategy to look for patterns.

- **Parallel Voting vs. Sequential Correction:**
The fundamental way these models build their decision trees is completely opposite.
 - **Random Forest** builds 300 trees independently at the same time. It averages their answers to get a stable, global prediction. It looks at the big picture but can miss small, complex details.
 - **Gradient Boosting** and **XGBoost** build trees in a chain. Each new tree is specifically designed to correct the exact mistakes made by the previous tree. This allows them to focus intensely on hard-to-classify water samples, which often leads to higher accuracy.

- **The Data Splitting Randomness (Bagging):**

Random Forest uses a technique called **Bagging** (Bootstrap Aggregating). This means each of its 300 trees only sees a random piece of the dataset and a random selection of features (for example, one tree might only look at pH and Turbidity, while another looks at Sulfate and Hardness). Because they look at different pieces of the puzzle, their final merged answer will naturally differ from a boosting algorithm.

- **Advanced Math and Regularization in XGBoost:**

XGBoost often performs differently (and usually better) than standard **Gradient Boosting** because it contains extra mathematical protection rules called **Regularization** (L_1 and L_2 penalties). These rules act like a speed limit: they penalize the model if it tries to become too complex or if it tries to memorize the training rows perfectly. This allows XGBoost to stay flexible and make better, smarter guesses on the hidden validation set (`X_val`).

- **Sensitivity to Extreme Values :**

Water data can be highly unstable. For example, a sample can have a normal pH but an extremely high amount of Solids.

- Random Forest is very robust and is not easily disturbed by extreme values because the majority vote of 300 trees dampens the error.
- Boosting models (Gradient Boosting and XGBoost) try so hard to fix every mistake that they might focus too much on a weird, extreme outlier sample, changing their internal boundaries.

6.1 Analyzing the Modest Performance: Why are the Scores around 66% to 68%?

Looking closely at our final results, we can observe that the global accuracy for all models ranges between 0.660 and 0.679 (66% to 68%), and the ROC-AUC scores are around 0.65 to 0.66. While these models are definitely learning patterns (since random guessing would give a score of 50%), these numbers are not exceptionally high.

There are different points to analyze why our classifiers are facing difficulties and hitting a performance ceiling:

1. **The Hidden Data Clashing :**

The primary reason for these scores is that water potability is not simple. In this dataset, many water samples have almost identical chemical features (same pH, same Sulfate, same Hardness), yet one sample is labeled as safe (1) and another is labeled as unsafe (0). When the data points overlap heavily in the 9-dimensional space, tree-based algorithms struggle to draw a clean boundary line without making mistakes.

2. **The Low Recall for Class 1 (Safe Water):**

If we look at the **Classification Report** for all models, there is a massive difference between the **Recall** of Class 0 and Class 1:

- **Class 0 (Unsafe):** The recall is very high (around 0.89 to 0.90). This means our models are excellent at catching contaminated water.

- **Class 1 (Safe):** The recall is quite low (around 0.29 to 0.34). This means our models are missing a lot of safe water samples, often predicting them as unsafe.

This happens because the dataset is somewhat unbalanced (400 unsafe samples versus only 256 safe samples in the validation set). The models play it safe by predicting "Unsafe" more often, which protects public health but lowers the overall accuracy score.

3. Missing Critical Chemical Features:

The dataset provides 9 features, which are great indicators, but water safety often depends on biological elements that are completely missing from our columns. For example, a water sample might have perfect pH, perfect Hardness, and perfect chemical scores, but if it contains microscopic bacteria, viruses, or heavy metals (like lead or arsenic) that are not tracked in our features, it is highly toxic (0). The model only sees the 9 clean columns and cannot guess these hidden biological dangers.

6.2 Privacy Issues, Ethical Risks, and Avoiding Misleading Claims

When working with Machine Learning, especially on a project that affects human health like water safety, we cannot just look at accuracy scores. We must also think deeply about the ethical responsibilities of our work. If an AI gives a wrong prediction about a movie recommendation, it is not a big deal. But if our water model makes a mistake, people could get sick.

- **Privacy Issues and Data Anonymization:**

Fortunately, our water dataset contains purely environmental and physical measurements (such as pH, Sulfate, and Hardness). It does not contain any Personally Identifiable Information. Because the data is completely anonymous and public, there is zero risk of violating anyone's personal privacy or leaking sensitive data.

- **The Ethical Risk of False Negatives:**

In medical or health AI, a **False Negative** is the most dangerous error. In our project, a False Negative means the model looks at highly contaminated water and confidently predicts: *"This water is safe (1)"*. If a user trusts this blind prediction, they might drink toxic water. Because our models have an accuracy of 68%, relying blindly on the AI poses a real ethical risk.

- **How the Group Avoided Misleading Claims (Transparency):**

To solve these ethical risks, our group decided to be completely transparent. We avoided making any "marketing" claims like *"Our AI guarantees 100% safe water"*. Instead, we took the following concrete actions in our final report and web interface:

1. **Displaying the Limitations Clearly:** On our application interface, we explicitly show our accuracy (68%) and explain that the model cannot detect hidden bacteria or dangerous heavy metals like lead.

7 Feature Engineering (Web GUI)

7.1 Model Serialization and Export

To bridge the gap between our analytical research in the Jupyter Notebook and the end-user application, our trained Machine Learning models must be exported into a persistent format. For this critical step, we chose the `joblib` Python library.

While the standard `pickle` module is often used for object serialization in Python, `joblib` is specifically optimized for objects that carry large NumPy arrays internally, which is exactly the case for Scikit-Learn models like Random Forest or Gradient Boosting. It provides a lightweight, highly efficient way to save our optimized algorithms.

The exportation code at the end of our Notebook is straightforward and automates the creation of our embedded model files:

```
1 # Save models as .pkl files
2 for name, model in models.items():
3     filename = name.replace(" ", "_").lower() + ".pkl"
4     filepath = os.path.join("web/model", filename)
5
6     joblib.dump(model, filepath)
7     print(f"Saving Models : {filepath}")
```

This loop ensures that each trained model is systematically named and safely stored in our web application directory, ready to be deployed.

7.2 Deployment Phase: Streamlit

An AI model is only as valuable as its accessibility to field workers. To deploy our models, we chose Streamlit, an open-source Python framework designed specifically for Machine Learning and Data Science applications.

Streamlit was selected for several strategic reasons:

- **Ease of Use & Rapid Prototyping:** It allows data scientists to build interactive web interfaces purely in Python, without needing advanced knowledge of front-end languages (HTML/CSS/JavaScript).
- **Scalability & Integration:** It is perfectly suited for this kind of "simple input/output" triage system, yet robust enough to integrate complex deep learning models or advanced interactive graphics in the future.
- **Hybrid Deployment (Cloud & Local):** This is a crucial feature for the WaterRangers project. Because our pre-trained models are embedded directly within the app's files, the application can run 100% locally and offline. Field workers in remote areas without internet access can run the app directly on a standard laptop. Simultaneously, the exact same code can be seamlessly pushed to cloud servers (like Streamlit Community Cloud or AWS) for global online access.

7.3 Web Application Architecture

The deployment side of our project is entirely contained within the `web/` folder. The architecture is designed to clearly separate the business logic from the visual assets and data storage. The structure is as follows:

- **app.py:** This is the core script containing the entire logic of the Streamlit application (page configuration, input forms, model loading, and prediction display).
- **model/:** This directory houses our embedded Machine Learning models exported from the Notebook (`random_forest.pkl`, `gradient_boosting.pkl`, and `xgboost.pkl`).
- **assets/:** This folder contains all the visual elements (such as the KMUTT and ESIEA logos, and the application icon). Keeping these isolated ensures a clean codebase where UI resources do not clutter the functional logic.
- **data/:** A dynamic folder created by the application to store user-submitted data locally (in a CSV file). This allows the application to maintain a historical log of all water analyses performed through the interface.

7.4 Application Logic

The core of the application resides in `app.py`. Instead of presenting a monolithic block of code, it is more instructive to break down into its five logical components.

Step 1: Imports and Global Configuration

The script begins by importing the necessary libraries and configuring the Streamlit page. A critical element here is the use of `st.session_state`. Streamlit reruns the entire script from top to bottom every time a user interacts with a widget. By initializing a session state, we ensure that the application "remembers" that an analysis has been completed, preventing the interface from resetting unexpectedly.

```

1 import streamlit as st
2 import pandas as pd
3 import os
4 import joblib
5 from datetime import datetime
6
7 # 1. Configuration
8 st.set_page_config(
9     page_title="WaterRangers",
10    page_icon="assets/icon.png",
11    layout="centered"
12 )
13
14 # Initialize session state to keep track of analysis status
15 if "analysis_done" not in st.session_state:
16     st.session_state.analysis_done = False

```

Step 2: User Interface and Branding

To provide a professional and localized feel, the header integrates the logos of the partnering institutions (KMUTT and ESIEA). Streamlit's `st.columns` function is used to create a clean, responsive layout that aligns the images and the main title horizontally.

```

1 # 2. Header and Logos
2 col_logo1, col_titre, col_logo2 = st.columns([2, 4, 2])
3
4 with col_logo1:

```

```

5     if os.path.exists("assets/logo_kmutt.webp"):
6         st.image("assets/logo_kmutt.webp", width=160)
7
8 with col_logo2:
9     if os.path.exists("assets/logo_esiea.png"):
10        st.image("assets/logo_esiea.png", width=160)
11
12 st.title("Water Rangers")
13 st.markdown("Enter the sample parameters below to evaluate potability.
    All fields are required.")

```

Step 3: Data Collection (The Input Form)

To prevent the models from running prematurely every time a single number is adjusted, the 9 physicochemical inputs are grouped inside an `st.form`. This ensures that the data is only sent to the processing engine when the user explicitly clicks the "Run Analysis" button. The inputs are divided into three columns for better readability.

```

1 # 3. Input Form
2 with st.form("prediction_form"):
3     st.subheader("Sample Characteristics")
4
5     c1, c2, c3 = st.columns(3)
6
7     with c1:
8         ph = st.number_input("pH Level (0-14)", min_value=0.0, max_value
9 =14.0, value=9.45)
10        hardness = st.number_input("Hardness (mg/L)", value=145.81)
11        solids = st.number_input("Solids / TDS (ppm)", value=13168.53)
12
13    with c2:
14        chloramines = st.number_input("Chloramines (ppm)", value=9.44)
15        sulfate = st.number_input("Sulfate (mg/L)", value=310.58)
16        conductivity = st.number_input("Conductivity ( S /cm)", value
17 =592.66)
18
19    with c3:
20        organic_carbon = st.number_input("Organic Carbon (ppm)", value
21 =8.61)
22        trihalomethanes = st.number_input("Trihalomethanes ( g /L)",
23 value=77.58)
24        turbidity = st.number_input("Turbidity (NTU)", value=3.88)
25
26    st.markdown("<br>", unsafe_allow_html=True)
27    submitted = st.form_submit_button("Run Analysis",
28 use_container_width=True)

```

Step 4: The Prediction Engine

Once the form is submitted, the application transitions from UI rendering to data processing. The user inputs are formatted into a Pandas DataFrame, the exact structure expected by Scikit-Learn. The three pre-trained models are then loaded via `joblib`, and their respective predictions are stored in the session state.

```

1 # 4. Prediction Logic
2 if submitted:

```



```

3 # Cache inputs in session state so they persist after rerun
4 st.session_state.inputs = {
5     "ph": ph, "Hardness": hardness, "Solids": solids,
6     "Chloramines": chloramines, "Sulfate": sulfate,
7     "Conductivity": conductivity, "Organic_carbon": organic_carbon,
8     "Trihalomethanes": trihalomethanes, "Turbidity": turbidity
9 }
10
11 # Convert in DataFrame for model input
12 input_data = pd.DataFrame([st.session_state.inputs])
13
14 # Loading Models
15 rf_model = joblib.load("model/random_forest.pkl")
16 gb_model = joblib.load("model/gradient_boosting.pkl")
17 xgb_model = joblib.load("model/xgboost.pkl")
18
19 # Do predictions and store in session state
20 st.session_state.pred_rf = rf_model.predict(input_data)[0]
21 st.session_state.pred_gb = gb_model.predict(input_data)[0]
22 st.session_state.pred_xgb = xgb_model.predict(input_data)[0]
23
24 st.session_state.analysis_done = True

```

Step 5: Result Visualization and Data Logging

The final phase handles output and data retention. It visually renders the predictions of the three models (highlighting in green for "Safe" and red for "Unsafe"). Furthermore, it offers field workers the option to save their contextualized data. If submitted, the application automatically creates or appends the data to a local `save.csv` file, laying the groundwork for future model retraining.

```

1 # 5. Display Results and Save Data
2 if st.session_state.analysis_done:
3
4     with st.container(border=True):
5         st.subheader("Analysis Results")
6
7         r1, r2, r3 = st.columns(3)
8
9         def render_result(col, model_name, pred):
10             status = "Safe" if pred == 1 else "Unsafe"
11             color = "#2e7d32" if pred == 1 else "#d32f2f"
12             col.markdown(f"**{model_name}**")
13             col.markdown(f"<h3 style='color: {color}; margin-top: 0px
14 ;'>{status}</h3>", unsafe_allow_html=True)
15
16             render_result(r1, "Random Forest", st.session_state.pred_rf)
17             render_result(r2, "Gradient Boosting", st.session_state.pred_gb)
18             render_result(r3, "XGBoost", st.session_state.pred_xgb)
19
20             st.markdown("---")
21
22             # Section to save records for future research
23             st.markdown("**Contribute to our research**")
24             st.caption("Note: Saving this record will help us train the next
25 version of our deep learning model.")

```

```

24     st.caption("WARNING: The accuracy of our models does not exceed
        68\%; this project is for educational purposes only, and it cannot
        detect the presence of hidden bacteria or heavy metals.")
25
26     location = st.text_input("Location (Optional)", placeholder="e.g
        ., Bangkok, Chao Phraya River")
27
28     if st.button("Save Record", type="primary"):
29         new_data = st.session_state.inputs.copy()
30         new_data["Location"] = location
31         new_data["Timestamp"] = datetime.now().strftime("%Y-%m-%d %H
        :%M:%S")
32         new_data["RF_Pred"] = st.session_state.pred_rf
33         new_data["GB_Pred"] = st.session_state.pred_gb
34         new_data["XGB_Pred"] = st.session_state.pred_xgb
35
36         df_new = pd.DataFrame([new_data])
37         file_path = "data/save.csv"
38
39         os.makedirs("data", exist_ok=True)
40
41         if os.path.exists(file_path):
42             df_new.to_csv(file_path, mode='a', header=False, index=
False)
43         else:
44             df_new.to_csv(file_path, mode='w', header=True, index=
False)
45
46         st.success("Record successfully saved to database. Thank you
        !")

```

7.5 Future Perspectives

The current version of the web application successfully achieves our primary goal: providing a fast, offline-capable prediction tool. However, the architecture we have built opens the door to several significant technical improvements.

Firstly, the local data saving feature (`save.csv`) is not just an administrative log; it is the foundation for a continuous learning pipeline. As field workers input new real-world data and verify the results, this growing, localized dataset can be used to periodically retrain our models. Over time, this feedback loop would allow the models to adapt specifically to the unique physicochemical signatures of local water sources, steadily increasing the current 68% accuracy ceiling.

Secondly, Streamlit's native capabilities allow for the seamless integration of interactive data visualization. In future iterations, we could easily add a dedicated "Data Analysis" tab to the application. This section would process the historical data stored in the CSV file, generating dynamic graphs (such as geographic mapping of pollution hotspots or time-series analysis of pH fluctuations). This would transform WaterRangers from a simple prediction tool into a comprehensive environmental monitoring dashboard.

8 Conclusion and Future Work

In conclusion, the Water Rangers project successfully created a fast, low-cost, and open-source "triage system" to predict water potability. By using powerful machine learning models (Random Forest, Gradient Boosting, and XGBoost), our system can analyze 9 physical and chemical water parameters in milliseconds.

Although the models face a performance ceiling of 66% to 68% accuracy due to overlapping data and missing biological features (like bacteria), they are excellent at catching contaminated water, with a recall score of nearly 90%. This prioritizes public health by safely identifying unsafe water. By deploying these models into an offline-capable Streamlit web application, we created a practical tool that field workers can use autonomously in remote areas without an internet connection.

For future work, we plan to improve this project through three main steps:

Continuous Learning: Using the local data saving feature (save.csv) to retrain models with new real-world data and improve regional accuracy.

Better Features: Finding ways to integrate missing biological indicators to bypass the current data limitations.

Data Dashboard: Adding a visual dashboard tab in Streamlit to map pollution hotspots and track water quality trends over time.

Ultimately, this project perfectly aligns with KMUTT's sustainable philosophy and the UN Sustainable Development Goal 6 (SDG 6), demonstrating how computer science can turn raw data into a positive social impact.

9 Team Contribution

The table below summarizes the exact contributions of each team member across all phases of the project, including software development, data analysis, report writing, and presentation preparation.

Table 1: Team Members Contribution Matrix

Phase / Task	Athenais	Nicolas	Alix
Coding & Tech	- Data Visualization - Data Analysis	- GitHub Repository setup - Created README.txt - Web GUI development	- Data Preprocessing - ML Models building - Models Training
Report Writing	- Data Description - Data Visualization & EDA - Conclusion	Introduction - Problem Definition & Goals - Feature Engineering (Web GUI)	- Use ML to solve the problem - Responsible AI Practices - Team Contribution
Slides & Pitch	- Associated slides & layout	- Associated slides & structure	- Associated slides & content

References

- [1] World Health Organization & United Nations Children’s Fund. (n.d.). *WHO/UNICEF Joint Monitoring Programme for water supply, sanitation and hygiene (JMP)*. Wash-Data. <https://washdata.org/>
- [2] United Nations Department of Economic and Social Affairs. (n.d.). *Thailand - SDG 6 progress*. United Nations. <https://sdgs.un.org/basic-page/thailand-34142>
- [3] King Mongkut’s University of Technology Thonburi. (n.d.). *Goal 6 - clean water and sanitation*. KMUTT Sustainability. <https://sustainable.kmutt.ac.th/goals-6/>
- [4] World Bank. (n.d.). *Testing water quality: When labs don’t work*. World Bank Blogs. <https://blogs.worldbank.org/en/water/when-labs-don-t-work>
- [5] Kaur, K., Singh, K., Gupta, S. K., & Tiwary, A. K. (2024). *Predicting water quality index using machine learning techniques: a case study of river Ganga in Haridwar, India*. ResearchGate. https://www.researchgate.net/publication/390638430_Predicting_water_quality_index_using_machine_learning_techniques_a_case_study_of_river_Ganga_in_Haridwar_India